Views should not be required to be default constructible

Document #:	P2325R2
Date:	2021-04-23
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

Since [P2325R1], added wording.

Since [P2325R0], added discussion of the different treatments of lvalue vs rvalue fixed-extent `span` in pipelines.

§ 2 Introduction

Currently, the `view` concept is defined in 24.4.4 [range.view] as:

```
template <class T>
concept view =
    range<T> &&
    movable<T> &&
    default_initializable<T> &&
    enable_view<T>;
```

Three of these four criteria, I understand. A `view` clearly needs to be a `range`, and it's important that they be `movable` for various operations to work. And the difference between a `view` and `range` is largely semantic, and so there needs to be an explicit opt-in in the form of `enable_view`.

But why does a view need to be `default_initializable`?

§ 2.1 History

The history of the design of Ranges is split between many papers and github issues in both the range-v3 [range-v3] and stl2 [stl2] libraries. However, I simply am unable to find much information that motivates this particular choice.

In [N4128], we have (this paper predates the term `view`, at the time the term "range" instead was used to refer to what is now called a `view`. To alleviate confusion, I have editted this paragraph accordingly):

> We've already decided that [Views] are copyable and assignable. They are, in the terminology of [EoP] and [N3351], Semiregular types. It follows that copies are independent, even though the copies are both aliases of the same underlying elements. The [views] are independent in the same way that a copy of a pointer or an iterator is independent from the original. Likewise, iterators from two [views] that are copies of each other are also independent. When the source [view] goes out of scope, it does not invalidate an iterator into the destination [view].
>
> Semiregular also requires DefaultConstructible in [N3351]. We follow suit and require all [Views] to be DefaultConstructible. Although this complicates the implementation of some range types, it has proven useful in practice, so we have kept this requirement.

There is also [stl2-179], titled "Consider relaxing the DefaultConstructible requirements," in which Casey Carter states (although the issue is about iterators rather than views):

> There's concern in the community that relaxing type invariants to allow for default construction of a type that would not otherwise provide it is a horrible idea.
>
> Relaxing the default construction requirement for iterators would also remove one of the few "breaking" differences between input and output iterators in the Standard (which do not require default construction) and Ranges (which currently do require default construction).

Though, importantly, Casey points out one concern:

> The recent trend of making everything in the standard library `constexpr` is in conflict with the desire to not require default construction. The traditional workaround for delayed initialization of a non-default-constructible `T` is to instead store an `optional<T>`. Changing an `optional<T>` from the empty to filled states is not possible in a constant expression

This was true at the time of the writing of the issue, but has since been resolved first at the core language level by [P1330R0] and then at the library level by [P2231R1]. As such, I'm simply unsure what the motivation is for requiring default construction of views.

The motivation for default construction of *iterators* comes from [N3644], although this doesn't really apply to *output* iterators (which are also currently required to be default constructible).

§ 2.2 Uses of default construction

I couldn't find any other motivation for default construction of views from the paper trail, so I tried to discover the motivation for it in range-v3. I did this with a large hammer: I removed all the default constructors and saw what broke.

And the answer is… not much. The commit can be found here: [range-v3-no-dflt]. The full list of breakage is:

1. `join_view` and `join_with_view` need a default-constructed inner view. This clearly breaks if that view isn't default constructible. I wrapped them in semiregular_box.

2. `views::ints` and `views::indices` are interesting in range-v3 because it's not just that `ints(0, 4)` gives you the range `[0,4)` but also that `ints` by itself is also a range (from `0` to infinity). These two inherit from iota, so once I removed the default constructor from iota, these uses break. So I added default constructors to `ints` and `indices`.

3. One of range-v3's mechanisms for easier implementation of views and iterators is called `view_facade`. This is an implementation strategy that uses the view as part of the iterator as an implementation detail. As such, because the iterator has to be default constructible, the view must be as well. So `linear_distribute_view` and `chunk_view` (the specialization for input ranges) kept their defaulted default constructors. But this is simply an implementation strategy, there's nothing inherent to these views that requires this approach.

4. There's one test for `any_view` that just tests that it's default constructible.

That's it. Broadly, just a few views that actually need default construction that can easily provide it, most simply don't need this constraint.

§ 2.3 Does this requirement cause harm?

Rather than providing a benefit, it seems like the default construction requirement causes harm.

If the argument for default construction is that it enables efficient deferred initialization during view composition, then I'm not sure I buy that argument. `join_view` would have to use an optional where it wouldn't have before, which makes it a little bigger. But conversely, right now, every range adaptor that takes a function has to use an optional: `transform_view`, `filter_view`, etc. all need to be default constructible so they have to wrap their callables in *`semiregular-box`* to make them default constructible. If views didn't have to be constructible, they wouldn't have to do this. Or rather, they would still have to do some wrapping, but we'd only need the assignment parts of *`semiregular-box`*, and not the default construction part, which means that `sizeof(`*`copyable-box`*`<T>)` would be equal to `sizeof(T)`, whereas `sizeof(`*`semiregular-box`*`<T>)` could be larger.

My impression right now is that the default construction requirement actually adds storage cost to range adapters on the whole rather than removing storage cost.

Furthermore, there's the question of *requiring* a partially formed state to types even they didn't want to do that. This goes against the general advice of making bad states unrepresentable. Consider a type like `span<int, 5>`. This *should* be a `view`: it's a non-owning, O(1)-everything range. But it's not default

constructible, so it's not a `view`. The consequence of this choice is the difference in behavior when using fixed-extent `span` in pipelines that start with an lvalue vs an rvalue:

```
std::span<int, 5> s = /* ... */;

// Because span<int, 5> is not a view, rather than copying s into
// the resulting transform_view, this instead takes a
// ref_view<span<int, 5>>. If s goes out of scope, this will dangle.
auto lvalue = s | views::transform(f);

// Because span<int, 5> is a borrowed range, this compiles. We still
// don't copy the span<int, 5> directly, instead we end up with a
// subrange<span<int, 5>::iterator>.
auto rvalue = std::move(s) | views::transform(f);
```

Both alternatives are less efficient than they could be. The lvalue case requires an extra indirection and exposes an opportunity for a dangling range. The rvalue case won't dangle, but ends up requiring storing two iterators, which requires twice the storage as storing the single `span` would have. Either case is strictly worse than the behavior that would result from `span<int, 5>` having been a `view`.

But fixed-extent `span` isn't default-constructible for good reason: if we were to add a default constructor that would make `span<int, 5>` partially formed, this adds an extra state that needs to be carefully checked by users, and suddenly every operation has additional preconditions that need to be documented. But this is true for every other view, too!

`ranges::ref_view` (see 24.7.4.2 [range.ref.view]) is another such view. In the same way that `std::reference_wrapper<T>` is a rebindable reference to `T`, `ref_view<R>` is a rebindable reference to the range `R`. Except `reference_wrapper<T>` isn't default constructible, but `ref_view<R>` is — it's just that as a user, I have no way to check to see if a particular `ref_view<R>` is fully formed or not. All of its member functions have this precondition that it really does refer to a range that I as the user can't check. This is broadly true of all the range adapters: you can't do *anything* with a default constructed range adapter except assign to it.

If the default construction requirement doesn't add benefit (and I'm not sure that it does) and it causes harm (both in the sense of requiring invalid states on types and adding to the storage requirements on all range adapters and further adding to user confusion when their types fail to model `view`), maybe we should get rid of it?

§ 3 Proposal

Remove the `default_initializable` constraint from `view`, such that the concept becomes:

```
template <class T>
concept view =
    range<T> &&
    movable<T> &&
    enable_view<T>;
```

Remove the `default_initializable` constraint from `weakly_incrementable`. This ends up removing the default constructible requirement from input-only and output iterators, while still keeping it on forward iterators (`forward_iterator` requires `incrementable` which requires `regular`).

For `iota_view`, replace the `semiregular<W>` constraint with `copyable<W>`, and add a constraint on `iota_view<W, Bound>::iterator`'s default constructor. This allows an input-only `iota_view` with a non-default-constructible `W` while preserving the current behavior for all forward-or-better `iota_views`.

Remove the default constructors from the standard library views and iterators for which they only exist to satisfy the requirement (`ref_view`, `istream_view`, `ostream_iterator`, `ostreambuf_iterator`, `back_insert_iterator`, `front_insert_iterator`, `insert_iterator`). Constrain the other standard library views' default constructors on the underlying types being default constructible.

For `join_view`, store the inner view in a `optional<views::all_t<InnerRng>>`.

Make `span<ElementType, Extent>` a `view` regardless of `Extent`. Currently, it is only a `view` when `Extent == 0 || Extent == dynamic_extent`.

We currently use *`semiregular-box`*`<T>` to make types `semiregular` (see 24.7.3 [range.semi.wrap]), which we use to wrap function objects throughout. We can do a little bit better by introducing a *`copyable-box`*`<T>` such that:

— If `T` is `copyable`, then *`copyable-box`*`<T>` is basically just `T`

— Otherwise, if `T` is `nothrow_copy_constructible` but not `copy_assignable`, then *`copyable-box`*`<T>` can be a thin wrapper around `T` that adds a copy assignment operator that does destroy-then-copy-construct.

— Otherwise, *`copyable-box`*`<T>` is *`semiregular-box`*`<T>` (we still need `optional<T>`'s empty state here to handle the case where copy construction can throw, to avoid double-destruction).

Replace all function object *`semiregular-box`*`<F>` wrappers throughout `<ranges>` with *`copyable-box`*`<F>` wrappers. At this point, there are no uses of *`semiregular-box`*`<F>` left, so remove it.

§ 3.1 Timeline

At the moment, only libstdc++ and MSVC provide an implementation of ranges (and MSVC's is incomplete). We either have to make this change now and soon, or never.

§ 4 Wording

Make `span` unconditionally a view in 22.7.2 [span.syn]:

```
namespace std {
  // ...

  // [views.span], class template span
  template<class ElementType, size_t Extent = dynamic_extent>
    class span;
```

```
  template<class ElementType, size_t Extent>
    inline constexpr bool ranges::enable_view<span<ElementType, Extent>> =
-     Extent == 0 || Extent == dynamic_extent;
+     true;

  // ...
}
```

Change 23.3.4.4 [iterator.concept.winc]/1:

```
template<class I>
  concept weakly_incrementable =
-   default_initializable<I> && movable<I> &&
+   movable<I> &&
    requires(I i) {
      typename iter_difference_t<I>;
      requires is-signed-integer-like<iter_difference_t<I>>;
      { ++i } -> same_as<I&>;   // not required to be equality-preserving
      i++;                      // not required to be equality-preserving
    };
```

Remove the default constructor and default member initializer from 23.5.2.2 [back.insert.iterator]:

```
namespace std {
  template<class Container>
  class back_insert_iterator {
  protected:
-   Container* container = nullptr;
+   Container* container;

  public:
    // ...

-   constexpr back_insert_iterator() noexcept = default;

    // ...
  };
}
```

Remove the default constructor and default member initializer from 23.5.2.3 [front.insert.iterator]:

```
namespace std {
  template<class Container>
  class front_insert_iterator {
  protected:
-   Container* container = nullptr;
+   Container* container;

  public:
    // ...

-   constexpr front_insert_iterator() noexcept = default;

    // ...
```

```
  };
}
```

Remove the default constructor and default member initializers from 23.5.2.4 [insert.iterator]:

```
namespace std {
  template<class Container>
  class insert_iterator {
  protected:
-   Container* container = nullptr;
-   ranges::iterator_t<Container> iter = ranges::iterator_t();
+   Container* container;
+   ranges::iterator_t<Container> iter;

  public:
    // ...

-   insert_iterator() = default;

    // ...
  };
}
```

Constrain the defaulted default constructor in 23.5.4.1 [common.iterator] [Editor's note: This is not strictly necessary since `variant<I, S>` is not default constructible when `I` is not and so the default constructor would already be defined as deleted, but I think it just adds clarity to do this for consistency]:

```
namespace std {
  template<input_or_output_iterator I, sentinel_for<I> S>
    requires (!same_as<I, S> && copyable<I>)
  class common_iterator {
  public:
-   constexpr common_iterator() = default;
+   constexpr common_iterator() requires default_initializable<I> = default;

    // ...

  private:
    variant<I, S> v_;   // exposition only
  };
```

Constrain the defaulted default constructor in 23.5.6.1 [counted.iterator]:

```
namespace std {
  template<input_or_output_iterator I>
  class counted_iterator {
  public:
    // ...

-   constexpr counted_iterator() = default;
+   constexpr counted_iterator() requires default_initializable<I> =
        default;

    // ...
```

```
  private:
    I current = I();                    // exposition only
    iter_difference_t<I> length = 0;    // exposition only
  };

  // ...
}
```

Remove the default constructor and default member initializers from 23.6.3.1 [ostream.iterator.general]:

```
namespace std {
  template<class T, class charT = char, class traits = char_traits<charT>>
  class ostream_iterator {
  public:
    // ...

-   constexpr ostream_iterator() noexcept = default;

    // ...

  private:
-   basic_ostream<charT,traits>* out_stream = nullptr;          //
        exposition only
-   const charT* delim = nullptr;                               //
        exposition only
+   basic_ostream<charT,traits>* out_stream;                    //
        exposition only
+   const charT* delim;                                         //
        exposition only
  };
}
```

Remove the default constructor and default member initializer from 23.6.5.1 [ostreambuf.iterator.general]:

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
  class ostreambuf_iterator {
  public:
    // ...

-   constexpr ostreambuf_iterator() noexcept = default;

    // ...

  private:
-   streambuf_type* sbuf_ = nullptr;    // exposition only
+   streambuf_type* sbuf_;              // exposition only
  };
}
```

Adjust the `iota_view` constraints in 24.2 [ranges.syn]:

```
#include <compare>              // see [compare.syn]
#include <initializer_list>     // see [initializer.list.syn]
#include <iterator>             // see [iterator.synopsis]
```

```
namespace std::ranges {
  // ...

  // [range.iota], iota view
  template<weakly_incrementable W, semiregular Bound =
        unreachable_sentinel_t>
-   requires weakly-equality-comparable-with<W, Bound> && semiregular<W>
+   requires weakly-equality-comparable-with<W, Bound> && copyable<W>
  class iota_view;

  template<class W, class Bound>
    inline constexpr bool enable_borrowed_range<iota_view<W, Bound>> = true;

  namespace views { inline constexpr unspecified iota = unspecified; }

  // ...
}
```

Remove `default_initializable` from the `view` concept in 24.4.4 [range.view]/1:

```
template<class T>
  concept view =
-   range<T> && movable<T> && default_initializable<T> && enable_view<T>;
+   range<T> && movable<T> && enable_view<T>;
```

Constrain the defaulted default constructor in 24.5.4.1 [range.subrange.general]:

```
namespace std::ranges {
  // ...

  template<input_or_output_iterator I, sentinel_for<I> S = I, subrange_kind
        K =
      sized_sentinel_for<S, I> ? subrange_kind::sized :
        subrange_kind::unsized>
    requires (K == subrange_kind::sized || !sized_sentinel_for<S, I>)
  class subrange : public view_interface<subrange<I, S, K>> {
  private:
    static constexpr bool StoreSize =                          //
        exposition only
      K == subrange_kind::sized && !sized_sentinel_for<S, I>;
    I begin_ = I();                                            //
        exposition only
    S end_ = S();                                              //
        exposition only
    make-unsigned-like-t<iter_difference_t<I>> size_ = 0;      //
        exposition only; present only
                                                               // when
        StoreSize is true
  public:
-   subrange() = default;
+   subrange() requires default_initializable<I> = default;

    // ...
  };
```

```
  // ...
}
```

Constrain the defaulted default constructor and switch to *copyable-box* in 24.6.3.2 [range.single.view]:

```
namespace std::ranges {
  template<copy_constructible T>
    requires is_object_v<T>
  class single_view : public view_interface<single_view<T>> {
  private:
-   semiregular-box<T> value_;   // exposition only (see [range.semi.wrap])
+   copyable-box<T> value_;      // exposition only (see [range.copy.wrap])
  public:
-   single_view() = default;
+   single_view() requires default_initializable<T> = default;

    // ...
  };
}
```

Change the synopsis in 24.6.4.2 [range.iota.view]:

```
  template<weakly_incrementable W, semiregular Bound =
        unreachable_sentinel_t>
-   requires weakly-equality-comparable-with<W, Bound> && semiregular<W>
+   requires weakly-equality-comparable-with<W, Bound> && copyable<W>
  class iota_view : public view_interface<iota_view<W, Bound>> {
  private:
    // [range.iota.iterator], class iota_view::iterator
    struct iterator;            // exposition only
    // [range.iota.sentinel], class iota_view::sentinel
    struct sentinel;            // exposition only
    W value_ = W();             // exposition only
    Bound bound_ = Bound();     // exposition only
  public:
-   iota_view() = default;
+   iota_view() requires default_initializable<W> = default;
    constexpr explicit iota_view(W value);
    constexpr iota_view(type_identity_t<W> value,
                        type_identity_t<Bound> bound);
    constexpr iota_view(iterator first, sentinel last) : iota_view(*first,
        last.bound_) {}

    constexpr iterator begin() const;
    constexpr auto end() const;
    constexpr iterator end() const requires same_as<W, Bound>;

    constexpr auto size() const requires see below;
  };
```

Constrain the defaulted default constructor and adjust the constraint in 24.6.4.3 [range.iota.iterator]:

```
namespace std::ranges {
  template<weakly_incrementable W, semiregular Bound>
-   requires weakly-equality-comparable-with<W, Bound> && semiregular<W>
+   requires weakly-equality-comparable-with<W, Bound> && copyable<W>
```

```
  struct iota_view<W, Bound>::iterator {
  private:
    W value_ = W();             // exposition only
  public:
    using iterator_concept = see below;
    using iterator_category = input_iterator_tag;       // present only if W
        models incrementable
    using value_type = W;
    using difference_type = IOTA-DIFF-T(W);

-   iterator() = default;
+   iterator() requires default_initializable<W> = default;

    // ...
  };
}
```

Adjust the constraint in 24.6.4.4 [range.iota.sentinel]:

```
namespace std::ranges {
  template<weakly_incrementable W, semiregular Bound>
-   requires weakly-equality-comparable-with<W, Bound> && semiregular<W>
+   requires weakly-equality-comparable-with<W, Bound> && copyable<W>
  struct iota_view<W, Bound>::sentinel {
    // ...
  };
}
```

Remove the default constructor and default member initializers, as well as the `if` from 24.6.5.2 [range.istream.view]:

```
namespace std::ranges {
  // ...

  template<movable Val, class CharT, class Traits>
    requires default_initializable<Val> &&
             stream-extractable<Val, CharT, Traits>
  class basic_istream_view : public view_interface<basic_istream_view<Val,
        CharT, Traits>> {
  public:
-   basic_istream_view() = default;
    constexpr explicit basic_istream_view(basic_istream<CharT, Traits>&
        stream);

    constexpr auto begin()
    {
-     if (stream_) {
        *stream_ >> value_;
-     }
      return iterator{*this};
    }

    constexpr default_sentinel_t end() const noexcept;

  private:
    struct iterator;                                    // exposition only
```

```
-   basic_istream<CharT, Traits>* stream_ = nullptr;    // exposition only
-   Val value_ = Val();                                 // exposition only
+   basic_istream<CharT, Traits>* stream_;              // exposition only
+   Val value_;                                         // exposition only
  };
}
```

Remove the default constructor and default member initializer from 24.6.5.3 [range.istream.iterator]:

```
namespace std::ranges {
  template<movable Val, class CharT, class Traits>
    requires default_initializable<Val> &&
             stream-extractable<Val, CharT, Traits>
  class basic_istream_view<Val, CharT, Traits>::iterator {      //
       exposition only
  public:
    // ...

-   iterator() = default;

    // ...

  private:
-   basic_istream_view* parent_ = nullptr;                      //
       exposition only
+   basic_istream_view* parent_;                                //
       exposition only
  };
}
```

Remove having to handle the case where `parent_ == nullptr` from all the iterator operations in 24.6.5.3 [range.istream.iterator] as this state is no longer representable:

```
iterator& operator++();
```

2 ~~Preconditions: `parent_->stream_ != nullptr` is `true`.~~

3 Effects: Equivalent to: `*parent_->stream_ >> parent_->value_;` `return *this;`

```
void operator++(int);
```

4 ~~*Preconditions:* `parent_->stream_ != nullptr` is `true`.~~

5 *Effects*: Equivalent to ++*this.

```
Val& operator*() const;
```

6 ~~*Preconditions:* `parent_->stream_ != nullptr` is `true`.~~

7 *Effects*: Equivalent to: return `parent_->value_;`

```
friend bool operator==(const iterator& x, default_sentinel_t);
```

8 *Effects*: Equivalent to: `return x.parent_ == nullptr || !*x.parent_->stream_;`

Replace the subclause [range.semi.wrap] (all the uses of `semiregular-box<T>` are removed with this paper) with a new subclause "Copyable wrapper" with stable name [range.copy.wrap]. The following is presented as a diff against the current 24.7.3 [range.semi.wrap], and also resolves [LWG3479]:

1 Many types in this subclause are specified in terms of an exposition-only class template ~~*semiregular-box*~~ *copyable-box*. ~~*semiregular-box<T>*~~ *copyable-box<T>* behaves exactly like `optional<T>` with the following differences:

(1.1) — ~~*semiregular-box<T>*~~ *copyable-box<T>* constrains its type parameter `T` with `copy_constructible<T>` && `is_object_v<T>`.

(1.2) — ~~If T models default_initializable, the~~ The default constructor of ~~*semiregular-box<T>*~~ *copyable-box<T>* is equivalent to:

```
- constexpr semiregular-box()
          noexcept(is_nothrow_default_constructible_v<T>)
-     : semiregular-box{in_place}
+ constexpr copyable-box() noexcept(is_nothrow_default_constructible_v<T>)
          requires default_initializable<T>
+     : copyable-box{in_place}
{ }
```

(1.3) — If `copyable<T>` is not modeled, the copy assignment operator is equivalent to:

```
- semiregular-box& operator=(const semiregular-box& that)
+ copyable-box& operator=(const copyable-box& that)
    noexcept(is_nothrow_copy_constructible_v<T>)
  {
+   if (this != addressof(that)) {
      if (that) emplace(*that);
      else reset();
+   }
    return *this;
  }
```

(1.4) — If `movable<T>` is not modeled, the move assignment operator is equivalent to:

```
- semiregular-box& operator=(semiregular-box&& that)
+ copyable-box& operator=(copyable-box&& that)
    noexcept(is_nothrow_move_constructible_v<T>)
  {
+   if (this != addressof(that)) {
      if (that) emplace(std::move(*that));
      else reset();
+   }
    return *this;
  }
```

2 *Recommended Practice*: *copyable-box<T>* should just store a `T` if either `T` models `copyable` or `is_nothrow_copy_constructible_v<T> && is_nothrow_copy_constructible_v<T>` is `true`.

Remove the default constructor and default member initializer from 24.7.4.2 [range.ref.view]:

1 `ref_view` is a `view` of the elements of some other `range`.

```
namespace std::ranges {
  template<range R>
    requires is_object_v<R>
  class ref_view : public view_interface<ref_view<R>> {
  private:
-   R* r_ = nullptr;            // exposition only
+   R* r_;                      // exposition only
  public:
-   constexpr ref_view() noexcept = default;

    // ...
  };
}
```

Constrain the defaulted default constructor and switch to *`copyable-box`* in 24.7.5.2 [range.filter.view]:

```
namespace std::ranges {
  template<input_range V, indirect_unary_predicate<iterator_t<V>> Pred>
    requires view<V> && is_object_v<Pred>
  class filter_view : public view_interface<filter_view<V, Pred>> {
  private:
    V base_ = V();                // exposition only
-   semiregular-box<Pred> pred_;  // exposition only
+   copyable-box<Pred> pred_;     // exposition only

    // [range.filter.iterator], class filter_view::iterator
    class iterator;                     // exposition only
    // [range.filter.sentinel], class filter_view::sentinel
    class sentinel;                     // exposition only

  public:
-   filter_view() = default;
+   filter_view() requires default_initializable<V> &&
        default_initializable<Pred> = default;
    constexpr filter_view(V base, Pred pred);

    // ...
  };

  // ...
}
```

Constrain the defaulted default constructor in 24.7.5.3 [range.filter.iterator]:

```
namespace std::ranges {
  template<input_range V, indirect_unary_predicate<iterator_t<V>> Pred>
    requires view<V> && is_object_v<Pred>
  class filter_view<V, Pred>::iterator {
  private:
    iterator_t<V> current_ = iterator_t<V>();   // exposition only
    filter_view* parent_ = nullptr;             // exposition only
  public:
    // ...
```

```
-   iterator() = default;
+   iterator() requires default_initializable<iterator_t<V>> = default;

    // ...
  };
}
```

Constrain the defaulted default constructor and switch to *`copyable-box`* in 24.7.6.2 [range.transform.view]:

```
namespace std::ranges {
  template<input_range V, copy_constructible F>
    requires view<V> && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<V>> &&
             can-reference<invoke_result_t<F&, range_reference_t<V>>>
  class transform_view : public view_interface<transform_view<V, F>> {
  private:
    // [range.transform.iterator], class template transform_view::iterator
    template<bool> struct iterator;             // exposition only
    // [range.transform.sentinel], class template transform_view::sentinel
    template<bool> struct sentinel;             // exposition only

    V base_ = V();            // exposition only
-   semiregular-box<F> fun_;  // exposition only
+   copyable-box<F> fun_;     // exposition only

  public:
-   transform_view() = default;
+   transform_view() requires default_initializable<V> &&
        default_initializable<F> = default;

    // ...
  };

  // ...
}
```

Constrain the defaulted default constructor in 24.7.6.3 [range.transform.iterator]:

```
namespace std::ranges {
  template<input_range V, copy_constructible F>
    requires view<V> && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<V>> &&
             can-reference<invoke_result_t<F&, range_reference_t<V>>>
  template<bool Const>
  class transform_view<V, F>::iterator {
  private:
    using Parent = maybe-const<Const, transform_view>;          //
        exposition only
    using Base = maybe-const<Const, V>;                         //
        exposition only
    iterator_t<Base> current_ = iterator_t<Base>();             //
        exposition only
    Parent* parent_ = nullptr;                                  //
        exposition only
```

```
  public:
    // ...

-   iterator() = default;
+   iterator() requires default_initializable<iterator_t<Base>> = default;

    // ...
  };
}
```

Constrain the defaulted default constructor in 24.7.7.2 [range.take.view]:

```
namespace std::ranges {
  template<view V>
  class take_view : public view_interface<take_view<V>> {
  private:
    V base_ = V();                                      // exposition only
    range_difference_t<V> count_ = 0;                   // exposition only
    // [range.take.sentinel], class template take_view::sentinel
    template<bool> struct sentinel;                     // exposition only
  public:
-   take_view() = default;
+   take_view() requires default_initializable<V> = default;

    // ...
  };

  // ...
}
```

Constrain the defaulted default constructor and switch to *copyable-box* in 24.7.8.2 [range.take.while.view]:

```
namespace std::ranges {
  template<view V, class Pred>
    requires input_range<V> && is_object_v<Pred> &&
             indirect_unary_predicate<const Pred, iterator_t<V>>
  class take_while_view : public view_interface<take_while_view<V, Pred>> {
    // [range.take.while.sentinel], class template take_while_view::sentinel
    template<bool> class sentinel;                      // exposition only

    V base_ = V();                 // exposition only
-   semiregular-box<Pred> pred_;   // exposition only
+   copyable-box<Pred> pred_;      // exposition only

  public:
-   take_while_view() = default;
+   take_while_view() requires default_initializable<V> &&
        default_initializable<Pred> = default;

    // ...
  };

  // ...
}
```

Constrain the defaulted default constructor in 24.7.9.2 [range.drop.view]:

```
namespace std::ranges {
  template<view V>
  class drop_view : public view_interface<drop_view<V>> {
  public:
-   drop_view() = default;
+   drop_view() requires default_initializable<V> = default;

    // ...
  private:
    V base_ = V();                              // exposition only
    range_difference_t<V> count_ = 0;           // exposition only
  };

  // ...
}
```

Constrain the defaulted default constructor and switch to *copyable-box* in 24.7.10.2 [range.drop.while.view]:

```
namespace std::ranges {
  template<view V, class Pred>
    requires input_range<V> && is_object_v<Pred> &&
             indirect_unary_predicate<const Pred, iterator_t<V>>
  class drop_while_view : public view_interface<drop_while_view<V, Pred>> {
  public:
-   drop_while_view() = default;
+   drop_while_view() requires default_initializable<V> &&
        default_initializable<Pred> = default;

    // ...

  private:
    V base_ = V();                 // exposition only
-   semiregular-box<Pred> pred_;   // exposition only
+   copyable-box<Pred> pred_;      // exposition only
  };

  template<class R, class Pred>
    drop_while_view(R&&, Pred) -> drop_while_view<views::all_t<R>, Pred>;
}
```

Constrain the defaulted default constructor and switch to `optional` in 24.7.11.2 [range.join.view] [Editor's note: This change conflicts with the proposed change in [P2328R0]. If that change is applied, only the constraint on the default constructor needs to be added]:

```
namespace std::ranges {
  template<input_range V>
    requires view<V> && input_range<range_reference_t<V>> &&
             (is_reference_v<range_reference_t<V>> ||
              view<range_value_t<V>>)
  class join_view : public view_interface<join_view<V>> {
  private:
    using InnerRng =                    // exposition only
```

```
      range_reference_t<V>;
    // [range.join.iterator], class template join_view::iterator
    template<bool Const>
      struct iterator;                  // exposition only
    // [range.join.sentinel], class template join_view::sentinel
    template<bool Const>
      struct sentinel;                  // exposition only

    V base_ = V();                      // exposition only
-   views::all_t<InnerRng> inner_ =     // exposition only, present only
        when !is_reference_v<InnerRng>
-     views::all_t<InnerRng>();
+   optional<views::all_t<InnerRng>> inner_;   // exposition only, present
        only when !is_reference_v<InnerRng>
  public:
-   join_view() = default;
+   join_view() requires default_initializable<V> = default;

    // ...
  };

  // ..
}
```

Constrain the defaulted default constructor in 24.7.11.3 [range.join.iterator]:

```
namespace std::ranges {
  template<input_range V>
    requires view<V> && input_range<range_reference_t<V>> &&
             (is_reference_v<range_reference_t<V>> ||
              view<range_value_t<V>>)
  template<bool Const>
  struct join_view<V>::iterator {
  private:
    using Parent    = maybe-const<Const, join_view>;            //
        exposition only
    using Base      = maybe-const<Const, V>;                    //
        exposition only
    using OuterIter = iterator_t<Base>;                         //
        exposition only
    using InnerIter = iterator_t<range_reference_t<Base>>;      //
        exposition only

    static constexpr bool ref-is-glvalue =                      //
        exposition only
      is_reference_v<range_reference_t<Base>>;

    OuterIter outer_ = OuterIter();                             //
        exposition only
    InnerIter inner_ = InnerIter();                             //
        exposition only
    Parent* parent_ = nullptr;                                  //
        exposition only

    constexpr void satisfy();                                   //
        exposition only
  public:
```

```
    // ...

-   iterator() = default;
+   iterator() requires default_initializable<OuterIter> &&
        default_initializable<InnerIter> = default;

    // ...
  };
}
```

Update the implementation parts of `join_view::iterator` now that `inner_` is an `optional` in 24.7.11.3 [range.join.iterator] (dereferencing `parent_->inner_` in both `satisfy()` and `operator++()`) [Editor's note: Don't apply these changes if [P2328R0] is adopted]:

```
constexpr void satisfy();       // exposition only
```

5 *Effects*: Equivalent to:

```
  auto update_inner = [this](range_reference_t<Base> x) -> auto& {
    if constexpr (ref-is-glvalue) // x is a reference
      return x;
    else
-     return (parent_->inner_ = views::all(std::move(x)));
+     parent_->inner_ = views::all(std::move(x));
+     return *parent_->inner_;
  };

  for (; outer_ != ranges::end(parent_->base_); ++outer_) {
    auto& inner = update_inner(*outer_);
    inner_ = ranges::begin(inner);
    if (inner_ != ranges::end(inner))
      return;
  }

  if constexpr (ref-is-glvalue)
    inner_ = InnerIter();
```

```
constexpr iterator& operator++();
```

9 Let *inner-range* be:

(9.1) — If *ref-is-glvalue* is `true`, `*outer_`.

(9.2) — Otherwise, `*parent_->inner_`.

10 *Effects*: Equivalent to:

```
auto&& inner_rng = inner-range;
if (++inner_ == ranges::end(inner_rng)) {
  ++outer_;
  satisfy();
}
return *this;
```

Constrain the defaulted default constructor in 24.7.12.2 [range.split.view] and change the implementation to use `optional<iterator_t<V>>` instead of a defaulted `iterator_t<V>` (same as `join`) [Editor's note: The same kind of change needs to be applied to [P2210R2] which is not yet in the working draft. The use of `optional` here should also be changed to *`non-propagating-cache`* with the adoption of [P2328R0].]:

```
namespace std::ranges {
  template<auto> struct require-constant;       // exposition only

  template<class R>
  concept tiny-range =                          // exposition only
    sized_range<R> &&
    requires { typename require-constant<remove_reference_t<R>::size()>; }
        &&
    (remove_reference_t<R>::size() <= 1);

  template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>,
        ranges::equal_to> &&
             (forward_range<V> || tiny-range<Pattern>)
  class split_view : public view_interface<split_view<V, Pattern>> {
  private:
    V base_ = V();                              // exposition only
    Pattern pattern_ = Pattern();               // exposition only
-   iterator_t<V> current_ = iterator_t<V>();   // exposition only, present
        only if !forward_range<V>
+   optional<iterator_t<V>> current_;           // exposition only, present
        only if !forward_range<V>
    // [range.split.outer], class template split_view::outer-iterator
    template<bool> struct outer-iterator;       // exposition only
    // [range.split.inner], class template split_view::inner-iterator
    template<bool> struct inner-iterator;       // exposition only
  public:
-   split_view() = default;
+   split_view() requires default_initializable<V> &&
        default_initializable<Pattern> = default;

    // ...
  };

  // ...
}
```

Change the description to dereference what is now an `optional` in 24.7.12.3 [range.split.outer]:

1 Many of the specifications in [range.split] refer to the notional member *`current`* of *`outer-iterator`*.*`current`* is equivalent to *`current_`* if `V` models `forward_range`, and *`*parent_->current_`* otherwise.

Constrain the defaulted default constructor in 24.7.14.2 [range.common.view]:

```
namespace std::ranges {
  template<view V>
    requires (!common_range<V> && copyable<iterator_t<V>>)
```

```
  class common_view : public view_interface<common_view<V>> {
  private:
    V base_ = V();  // exposition only
  public:
-   common_view() = default;
+   common_view() requires default_initializable<V> = default;

    // ...
  };

  // ...
}
```

Constrain the defaulted default constructor in 24.7.15.2 [range.reverse.view]:

```
namespace std::ranges {
  template<view V>
    requires bidirectional_range<V>
  class reverse_view : public view_interface<reverse_view<V>> {
  private:
    V base_ = V();  // exposition only
  public:
-   reverse_view() = default;
+   reverse_view() requires default_initializable<V> = default;

    // ...
  };

  // ...
}
```

Constrain the defaulted default constructor in 24.7.16.2 [range.elements.view]:

```
namespace std::ranges {
  // ...

  template<input_range V, size_t N>
    requires view<V> && has-tuple-element<range_value_t<V>, N> &&
             has-tuple-element<remove_reference_t<range_reference_t<V>>, N>
         &&
             returnable-element<range_reference_t<V>, N>
  class elements_view : public view_interface<elements_view<V, N>> {
  public:
-   elements_view() = default;
+   elements_view() requires default_initializable<V> = default;

    // ...

  private:
    // [range.elements.iterator], class template elements_view::iterator
    template<bool> struct iterator;                    // exposition only
    // [range.elements.sentinel], class template elements_view::sentinel
    template<bool> struct sentinel;                    // exposition only
    V base_ = V();                                     // exposition only
```

```
  };
}
```

Constrain the defaulted default constructor in 24.7.16.3 [range.elements.iterator]:

```
namespace std::ranges {
  template<input_range V, size_t N>
    requires view<V> && has-tuple-element<range_value_t<V>, N> &&
             has-tuple-element<remove_reference_t<range_reference_t<V>>, N>
         &&
             returnable-element<range_reference_t<V>, N>
  template<bool Const>
  class elements_view<V, N>::iterator {                 // exposition only
    using Base = maybe-const<Const, V>;                 // exposition only

    iterator_t<Base> current_ = iterator_t<Base>();     // exposition only

    static constexpr decltype(auto) get-element(const iterator_t<Base>& i);
        // exposition only

  public:
    // ...

-   iterator() = default;
+   iterator() requires default_initializable<iterator_t<Base>> = default;

    // ...
  };
}
```

§ 4.1 Feature-test macro

Bump the Ranges feature-test macro in 17.3.2 [version.syn]:

```
- #define __cpp_lib_ranges 201911L // also in <algorithm>, <functional>,
        <iterator>, <memory>, <ranges>
+ #define __cpp_lib_ranges 2021XXL // also in <algorithm>, <functional>,
        <iterator>, <memory>, <ranges>
```

§ 5 References

[EoP] Stepanov, A. and McJones, P. 2009. Elements of Programming. Addison-Wesley Professional.

[LWG3479] Casey Carter. semiregular-box mishandles self-assignment.
https://wg21.link/lwg3479

[N3351] B. Stroustrup, A. Sutton. 2012-01-13. A Concept Design for the STL.
https://wg21.link/n3351

[N3644] Alan Talbot. 2013-04-18. Null Forward Iterators.
https://wg21.link/n3644

[N4128] E. Niebler, S. Parent, A. Sutton. 2014-10-10. Ranges for the Standard Library, Revision 1.
https://wg21.link/n4128

[P1330R0] Louis Dionne, David Vandevoorde. 2018-11-10. Changing the active member of a union inside constexpr.
https://wg21.link/p1330r0

[P2210R2] Barry Revzin. 2021-03-05. Superior String Splitting.
https://wg21.link/p2210r2

[P2231R1] Barry Revzin. 2021. Missing `constexpr` in `std::optional` and `std::variant`.
https://wg21.link/p2231r1

[P2325R0] Barry Revzin. 2021-02-17. Views should not be required to be default constructible.
https://wg21.link/p2325r0

[P2325R1] Barry Revzin. 2021-03-16. Views should not be required to be default constructible.
https://wg21.link/p2325r1

[P2328R0] Tim Song. 2021-03-15. join_view should join all views of ranges.
https://wg21.link/p2328r0

[range-v3] Eric Niebler and Casey Carter. 2013. range-v3 repo.
https://github.com/ericniebler/range-v3/

[range-v3-no-dflt] Barry Revzin. 2021. Removing default construction from range-v3 views.
https://github.com/BRevzin/range-v3/commit/2e2c9299535211bc5417f9146eaed9945e596e83

[stl2] Eric Niebler and Casey Carter. 2014. stl2 repo.
https://github.com/ericniebler/stl2/

[stl2-179] Casey Carter. 2016. Consider relaxing the DefaultConstructible requirements.
https://github.com/ericniebler/stl2/issues/179